

Week6

Task 1

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>

#define SERVER_FIFO "/tmp/server_fifo"

int main() {
    char buffer[100];

    // Create server FIFO
    mkfifo(SERVER_FIFO, 0666);

    printf("Server started...\n");

    while (1) {
        int fd = open(SERVER_FIFO, O_RDONLY);

        if (fd == -1) {
            perror("open");
            exit(1);
        }

        // Read message
        ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);
        close(fd);

        if (bytes_read <= 0) {
            continue;
        }

        // Add null terminator
        buffer[bytes_read] = '\0';

        printf("Message received: %s\n", buffer);
    }
}
```

```

// Create a child process to handle each client
if (fork() == 0) {
    char response[120];

    printf("Processing client request...\n");

    snprintf(response, sizeof(response),
             "Server received: %s", buffer);

    // Print response
    printf("Response: %s\n", response);

    exit(0);
}

// Check for completed child processes
waitpid(-1, NULL, WNOHANG);
}

unlink(SERVER_FIFO);

return 0;
}

```

```

#include <fcntl.h>
#include <unistd.h>

#define SERVER_FIFO "/tmp/server_fifo"

int main() {
    char message[100];

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    int fd = open(SERVER_FIFO, O_WRONLY);

    if (fd == -1) {
        perror("open");
        exit(1);
    }

    write(fd, message, strlen(message) + 1);

    close(fd);

    printf("Message sent to server.\n");

    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K: ~
root@DESKTOP-VMN932K:~# rm server.c
root@DESKTOP-VMN932K:~# nano server.c
root@DESKTOP-VMN932K:~# gcc server.c -o server
root@DESKTOP-VMN932K:~# ./server
Server started...
Message received: Hi

Processing client request...
Response: Server received: Hi

```

```

root@DESKTOP-VMN932K: ~
root@DESKTOP-VMN932K:~# nano client.c
root@DESKTOP-VMN932K:~# gcc client.c -o client
root@DESKTOP-VMN932K:~# ./client
Enter message: Hi
Message sent to server.
root@DESKTOP-VMN932K:~# gcc client.c -o client

```

Task2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int sig) {

    if (sig == SIGINT) {
        printf("\nSIGINT received!\n");
        printf("This is usually generated by pressing Ctrl+C.\n");
    }

    else if (sig == SIGTERM) {
        printf("\nSIGTERM received!\n");
        printf("Termination request received.\n");
    }

    else if (sig == SIGUSR1) {
        printf("\nSIGUSR1 received!\n");
        printf("User-defined signal received.\n");
    }
}

int main() {

    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

Output:

root@DESKTOP-VMN932K: ~

Program is running...
Program is running...
Program is running...

SIGUSR1 received!
User-defined signal received.

Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...

SIGTERM received!
Termination request received.

Program is running...
Program is running...
Program is running...
Program is running...
Program is running...

SIGINT received!
This is usually generated by pressing Ctrl+C.

Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Killed

root@DESKTOP-VMN932K:~#

— U X

root@DESKTOP-VMN932K: ~

root@DESKTOP-VMN932K:~# kill -SIGUSR1 919
root@DESKTOP-VMN932K:~# kill -SIGTERM 919
root@DESKTOP-VMN932K:~# kill -SIGINT 919
root@DESKTOP-VMN932K:~# kill -9 919
root@DESKTOP-VMN932K:~#